Introduction

Every technical interview eventually circles back to one uncomfortable question: how do you store data when you can't afford to keep all of it? Arrays and lists are the default answer to "where do I put things," but they carry an assumption most people never question — that storage can always grow. Take that assumption away, and you're forced to think differently.

That's exactly the constraint embedded engineers live with. A wearable's microcontroller might have 64 kilobytes of RAM total, not gigabytes. A step-counting algorithm running on that chip still needs a rolling window of recent sensor readings, updated dozens of times a second, without ever asking the OS for more memory.

This is why the **ring buffer** (also called a circular buffer or circular queue) shows up constantly in interviews — it's one of the few data structures that's both a genuinely common LeetCode "design" problem (see `Design Circular Queue`, LeetCode 622) and something you'll actually implement on the job if you touch embedded systems, audio processing, networking, or logging infrastructure. Interviewers use it to test whether you can reason about pointer arithmetic, off-by-one edge cases, and space complexity simultaneously — not just whether you memorized an API.

By the end of this article, you'll understand not just how to implement a ring buffer, but why every design decision inside it exists.

Problem Overview

Design a fixed-capacity data structure that supports:

- **Enqueue(value)** — add a new value, evicting the oldest value automatically if the structure is full.
- **Dequeue()** — remove and return the oldest value.
- **Front()** / **Rear()** — peek at the oldest and newest values without removing them.
- **isEmpty()** / **isFull()** — report the current state.

The catch: the underlying storage must never resize. Whether you've inserted 10 values or 10 million, memory usage stays exactly the same. New values must overwrite the oldest ones once the buffer fills, and every operation must run in constant time — no shifting, no copying.

This is fundamentally different from a normal queue built on a Python list, where popping from the front is O(n) because everything behind it has to shift down.

Example

Imagine a ring buffer with capacity 5, tracking the last 5 heart-rate readings.

```
Insert: 72, 75, 74, 78, 80
Buffer: [72, 75, 74, 78, 80]   <- full
```

Now a new reading comes in: `76`.

```
Insert: 76
Buffer: [76, 75, 74, 78, 80]   <- 72 was overwritten
```

The oldest value (`72`) is silently discarded to make room. The buffer's size never changed — only its contents did. If you asked "what are the last 5 readings?" before and after, you'd get a correct, up-to-date answer both times, with zero memory reallocation in between.

Intuition

Start from the wrong-but-tempting solution: a plain Python list, where you `append()` new readings and `pop(0)` the oldest one once you exceed your window size. It's correct. It's also quietly expensive — `pop(0)` on a list is O(n) because every remaining element has to shift left by one. If you're sampling a sensor 100 times a second, you're doing that shift 100 times a second, forever.

The fix starts with a reframe: you don't actually need the list to shrink and grow. You need a **fixed number of slots** that get reused. Picture a physical row of exactly 5 slots on a shelf. You always write to the "next open" slot. When you run out of shelf space, you don't build a new shelf — you go back to the beginning and start overwriting, because the oldest item on the shelf is, by definition, the one you care about least.

That's the entire idea. Two pointers do the bookkeeping:

- **head** — points to the oldest value (what gets dequeued next).
- **tail** — points to the next empty slot (where the next value gets written).

Writing a value is `buffer[tail] = value`, then advancing tail. When tail walks off the end of the array, it needs to come back to index 0 — that's the "wraparound." Instead of an `if tail == capacity: tail = 0` check, you get the same result with one arithmetic expression:

```
tail = (tail + 1) % capacity
```

The modulo operator gives you the remainder after division. As long as `tail` is less than `capacity`, `tail % capacity` just returns `tail` unchanged. The moment `tail` would equal `capacity`, the modulo operation snaps it back to `0`. One line replaces a branch, and it works identically for `head` once the buffer is full and old entries start getting evicted.

Here's the part that trips almost everyone up the first time: what happens when `head` and `tail` land on the exact same index? That position alone is ambiguous — it means either the buffer is completely empty (nothing written yet) or completely full (every slot occupied, next write would overwrite the oldest unread value). The pointers can't tell you which. You need a third piece of state — typically a `size` counter — to break the tie.

Brute Force Solution

Idea: Use a plain Python list. Append new values to the end; once the list exceeds the window size, remove from the front.

```
window = []
window.append(new_reading)
if len(window) > WINDOW_SIZE:
    window.pop(0)
```

Advantages: - Trivial to write and reason about. - No pointer math, no edge cases around wraparound.

Disadvantages: - `pop(0)` is O(n) — every element shifts down one index. - Memory can spike temporarily during resizing operations under the hood. - Doesn't reflect how memory-constrained hardware actually needs to behave.

Time complexity: O(n) per insertion once the window is full. **Space complexity:** O(n) where n is the window size — same as the ring buffer, but the *time* cost is the real problem here.

Optimal Solution

The ring buffer solves the same problem with:

Component	Purpose
`buffer`	Fixed-size list, pre-allocated to `capacity`
`head`	Index of the oldest element

Component	Purpose
`tail`	Index of the next write position
`size`	Current count of elements (resolves the head==tail ambiguity)

Enqueue: 1. Write the new value at `buffer[tail]`. 2. Advance `tail = (tail + 1) % capacity`. 3. If the buffer was already full, advance `head` too (the oldest value is overwritten) — `size` stays at `capacity`. 4. Otherwise, increment `size`.

Dequeue: 1. Read the value at `buffer[head]`. 2. Advance `head = (head + 1) % capacity`. 3. Decrement `size`.

Every operation touches a fixed number of array slots and integer variables — no loops, no shifting, no reallocation. That's what makes it O(1).

Python Code

```
class RingBuffer:
    """Fixed-capacity circular buffer with O(1) enqueue/dequeue."""

    def __init__(self, capacity: int):
        if capacity <= 0:
            raise ValueError("capacity must be positive")
        self.capacity = capacity
        self.buffer = [None] * capacity
        self.head = 0
        self.tail = 0
        self.size = 0

    def is_empty(self) -> bool:
        return self.size == 0

    def is_full(self) -> bool:
        return self.size == self.capacity

    def enqueue(self, value) -> None:
        self.buffer[self.tail] = value
        self.tail = (self.tail + 1) % self.capacity
        if self.is_full():
            # Buffer was already full: the write above overwrote
            # the oldest slot, so head must advance too.
            self.head = (self.head + 1) % self.capacity
        else:
            self.size += 1

    def dequeue(self):
        if self.is_empty():
            raise IndexError("dequeue from empty buffer")
```

```
        value = self.buffer[self.head]
        self.buffer[self.head] = None
        self.head = (self.head + 1) % self.capacity
        self.size -= 1
        return value

    def front(self):
        if self.is_empty():
            raise IndexError("front of empty buffer")
        return self.buffer[self.head]

    def rear(self):
        if self.is_empty():
            raise IndexError("rear of empty buffer")
        return self.buffer[(self.tail - 1) % self.capacity]

    def to_list(self) -> list:
        """Oldest-to-newest snapshot of current contents."""
        return [self.buffer[(self.head + i) % self.capacity] for i in range(self.size)]
```

Code Walkthrough

- `__init__` : Pre-allocates the full array up front (`[None] * capacity`) — this is the "fixed memory" guarantee in code form. `head` , `tail` , and `size` all start at zero.
- `is_full` : Uses `size` , not pointer comparison, precisely because `head == tail` is ambiguous on its own.
- `enqueue` : Writes first, advances `tail` second. The `is_full()` check happens *after* the write but reflects state *before* the write — that ordering is what correctly detects "the buffer was already full when this call started."
- `dequeue` : Reads from `head` , clears the slot (optional, but avoids holding a stale reference), then advances `head` with the same wraparound formula.
- `rear` : Since `tail` always points to the *next* empty slot, the actual last-written value lives at `tail - 1` . The `% capacity` handles the case where `tail` is currently `0` (wraps to `capacity - 1`) — Python's modulo of a negative number stays non-negative, so this is safe without extra branching.
- `to_list` : Walks `size` elements starting at `head` , using modulo to stay in bounds regardless of where `head` currently sits in the underlying array.

Dry Run

Capacity 3. Start empty: `head=0, tail=0, size=0` .

```
enqueue(10)
  buffer = [10, None, None]
  tail = 1, size = 1 (not full yet)

enqueue(20)
  buffer = [10, 20, None]
  tail = 2, size = 2

enqueue(30)
  buffer = [10, 20, 30]
  tail = 0 (wrapped), buffer now full -> head advances to 1
  size stays 3

enqueue(40)
  buffer[0] = 40 -> buffer = [40, 20, 30]
  tail = 1
  already full -> head advances to 2
  size stays 3

to_list() reads starting at head=2:
  index 2 -> 30, index 0 -> 40, index 1 -> 20
  result: [30, 40, 20]
```

Notice `10` and `20`'s original slot got reused without ever resizing the list — and `to_list()` still returns values in correct oldest-to-newest order despite the physical array being "scrambled" by wraparound.

Complexity Analysis

- **Time — Enqueue/Dequeue: O(1).** Every operation performs a fixed number of index writes and modulo operations, regardless of how many elements have ever passed through the buffer.
- **Time — to_list(): O(n)** where n is `size`, since you're materializing a snapshot.
- **Space: O(capacity).** The array is allocated once, at construction, and never resized — this is the entire point of the structure, and it's exactly what makes it viable on memory-constrained hardware.

Alternative Solutions

Python's `collections.deque` with a `maxlen` argument gives you nearly identical behavior out of the box:

```
from collections import deque
window = deque(maxlen=5)
```

```
window.append(new_reading)  # oldest auto-evicted once full
```

`deque` is implemented as a doubly linked list of blocks internally, giving O(1) append/pop from both ends. For most application-level Python code, reach for `deque(maxlen=n)` — it's battle-tested and less code to maintain. The manual ring buffer earns its place when you need explicit control over the underlying array (embedded/C-like environments, custom memory layouts, or when an interviewer specifically wants to see you reason about pointers and modulo arithmetic instead of calling a library).

Edge Cases

1. **Empty buffer dequeue/front/rear** — must raise or signal an error rather than returning stale/`None` data silently.
2. **Capacity of 1** — `head` and `tail` collide immediately on the first enqueue; verify `is_full()` triggers correctly at size 1.
3. **Enqueue exactly `capacity` times, no more** — buffer should be full but head/tail should not have wrapped past their starting relationship incorrectly.
4. **Enqueue far beyond capacity** (e.g., 10x capacity) — confirms wraparound math holds under repeated cycling, not just on the first wrap.
5. **Duplicate values** — the structure doesn't care about value identity, only position; make sure equality checks in tests don't assume uniqueness.

Common Mistakes

1. **Using `head == tail` alone to mean "empty."** This silently misreports a full buffer as empty, corrupting every subsequent read.
2. **Forgetting the wraparound on `head`, not just `tail`.** Many implementations correctly wrap `tail` but forget `head` needs the same treatment once the buffer starts evicting.
3. **Off-by-one in `rear()`.** Since `tail` points to the *next* empty slot, forgetting to subtract 1 before reading returns garbage or `None`.
4. **Growing the array anyway.** Appending to a Python list instead of pre-allocating with `[None] * capacity` quietly reintroduces O(n) resizing behavior, defeating the purpose.
5. **Negative modulo assumptions from other languages.** In C, `-1 % 3` can be `-1`; in Python it's `2`. Code ported from another language may need adjustment.
6. **Not clearing evicted slots when it matters.** If buffer values hold large objects or sensitive data, leaving stale references in overwritten-but-unreached slots can cause memory leaks or data exposure in long-lived processes.

Interview Questions

- How would you make this thread-safe for a producer/consumer scenario?
- How does this compare to `collections.deque(maxlen=n)` under the hood?
- Could you implement this without an explicit `size` counter, using one wasted slot instead? (Yes — a common variant sacrifices one array slot so `head == tail` unambiguously means empty, and `(tail + 1) % capacity == head` means full.)
- How would you generalize this to support dynamic resizing when capacity needs change at runtime?
- What happens to this design under multiple concurrent producers?

Similar Problems

- **LeetCode 622 — Design Circular Queue**: Nearly identical to this implementation, testing the same head/tail/size mechanics directly.
- **LeetCode 641 — Design Circular Deque**: Extends the same idea to insertion/removal from both ends.
- **LeetCode 359 — Logger Rate Limiter**: Uses a similar fixed-window idea, though typically solved with a hash map rather than an array.
- **LeetCode 239 — Sliding Window Maximum**: A different sliding-window problem, but shares the "only care about the last k elements" framing.
- **LRU Cache (LeetCode 146)**: Different eviction policy (least-recently-used instead of oldest-inserted), but same underlying instinct — fixed capacity, constant-time operations, explicit eviction logic.

Key Takeaways

A ring buffer trades the illusion of infinite storage for something better in constrained environments: guaranteed, predictable memory usage combined with guaranteed, predictable speed. The wraparound trick — `(index + 1) % capacity` — replaces a conditional with arithmetic, and the head/tail/size trio resolves the one genuine ambiguity in the design. Once you've built one from scratch, you'll recognize the same pattern in audio buffers, network packet queues, and log rotation systems — anywhere a live stream meets a fixed memory budget.

Watch the Video

For the full walkthrough with visuals of the wraparound in motion, watch the video here: {LINK}

About the Series

This article is part of **Fun with Learning Technology**, a series that pulls apart the everyday technology you use — from the data structures inside your smartwatch to the systems powering the apps on your phone — and rebuilds the intuition from first principles. No shortcuts to "just memorize this," just the actual reasoning an engineer goes through to arrive at the solution.

Call To Action

If this helped the ring buffer finally click, drop a like on the video and subscribe on YouTube for more deep dives like this one. For the full written breakdowns before anyone else sees them, subscribe to the Substack. And if you've got a data structure or system you want cracked open next, leave it in the comments — that's genuinely how future topics get picked.

Circular Queues (Ring Buffers) for Streaming Sensor Data in Wearables — free Python cheat sheet